Module 4: How to Use AI Coding Agents for AI/ML Development

Live notebook:

```
marimo edit Module_4/4_1_ai_features_demo.py
```

4.1 When to Let AI Agents Write Code for You

Before the demos:

1. Open marimo notebook settings.
2. Install AI dependencies if prompted.
3. Open the **AI** tab.
4. Choose a hosted provider or local Ollama model.

Find the config file:

```
marimo config show
```

The path to `marimo.toml` is printed at the top of the output.

Demo: Generate with AI

Notebook section: **1. Generate with AI**

Prompt:

```
Using df, add a new cell that computes total revenue and total profit by
channel, sorted from highest to lowest revenue. Return a dataframe named
channel_summary.
```

Demo: Inline autocomplete

Notebook section: **2. Inline autocompletion**

In the empty cell under section 2, start typing:

```
high_margin = df[df["margin"] >
```

Let autocomplete finish the expression. If needed, complete it as:

```
high_margin = df[df["margin"] > 0.45]
high_margin
```

Demo: Refactor current cell

Notebook section: **3. Refactor the current cell**

Click into the repetitive `region_revenue_summary` cell and press `Ctrl/Cmd-Shift-E`.

Prompt:

```
Refactor this cell into a helper function named summarize_region_metric
that
takes df and metric as inputs, then returns the same region-level summary.
Keep the output dataframe named region_revenue_summary.
```

4.2 Giving AI Coding Agents the Context They Need

Your code is only part of the story. For AI/ML work, the assistant also needs execution state, dataframe schemas, widget values, model outputs, and dependency information.

Context in marimo

marimo helps because notebooks are reactive and stateful:

- `@df` gives the assistant dataframe context.
- `@metric_selector` gives the current widget value.
- `@segment_selector` gives the selected segment values.
- Existing variables and cell dependencies make the notebook easier to inspect.

Demo: Chat with variable context

Notebook section: **4. Chat panel with variable context**

Chat panel modes:

- **Manual**: The model only sees what you type or explicitly attach. It cannot inspect the notebook on its own.
- **Ask**: The model can inspect the notebook and gather context, but it does not change cells.
- **Agent (beta)**: The model can inspect the notebook, edit cells, and run stale cells.

Prompt A:

```
What patterns do you see in this data?
```

Prompt B:

```
Using @df, @metric_selector, and @segment_selector, add a cell that
summarizes
the selected metric for the selected segments and creates a simple
matplotlib
bar chart.
```

Custom instructions and rules

Use custom rules in marimo settings to keep AI output consistent across prompts and providers:

```
Use pandas for tabular transformations.
Use matplotlib for plots.
Keep generated cells small.
Avoid reloading data that already exists in the notebook.
```

For external agents, keep project-level instructions short and concrete. Good instructions include:

- which notebook command to run
- which style to follow
- whether to prefer pandas, polars, matplotlib, or plotly
- whether to run `marimo check` after edits

Linting and safety loop

AI can write valid Python that is invalid marimo. Always check after AI edits:

```
uvx marimo check Module_4/4_1_ai_features_demo.py
```

Apply safe fixes when available:

```
uvx marimo check --fix Module_4/4_1_ai_features_demo.py
```

Strict final check:

```
uvx marimo check --strict Module_4/4_1_ai_features_demo.py
```

4.3 Choosing the Right AI Coding Agent Setup

The right setup depends on cost, speed, privacy, control, and task complexity. Start with the built-in editor features, then move to agent workflows when the task spans multiple cells or files.

Option A: Built-in marimo AI

Use for:

- generating one cell
- refactoring one cell
- asking notebook-level questions
- working with `@` variable context

Best when you want fast iteration inside the notebook.

Option B: Local models with Ollama

Use when privacy matters or you want local experimentation.

See:

```
Module_4/Ollama-Setup.md
```

Example marimo config:

```
[ai.models]
chat_model = "ollama/gemma3:1b"
edit_model = "ollama/gemma3:1b"
autocomplete_model = "ollama/gemma3:1b"

[ai.ollama]
base_url = "http://127.0.0.1:11434/v1"
```

marimo uses Ollama's OpenAI-compatible API, so the `/v1` suffix matters.

Option C: Generate a whole notebook

Use `marimo new` when you want a fresh notebook scaffold:

```
marimo new "create a marimo notebook with a small sales dataframe, a
dropdown for region, and a bar chart of revenue by segment"
```

The three advanced pieces

Once the built-in editor features are clear, introduce the newer agent-facing tools as three separate ideas:

1. **Agents / marimo pair**: connect an external coding agent to a live notebook.
2. **MCP**: expose notebook-aware tools or bring external tools into Chat.
3. **Skills / custom instructions**: give agents reusable behavior and project conventions.

1. Agents: marimo pair with an agent CLI

This is the important new workflow to teach.

`marimo pair` connects agent CLIs such as Claude Code, Codex, and OpenCode to a running marimo notebook. The agent can inspect variables, test logic, run cells, edit cells, and interact with UI elements.

Install:

```
npx skills add marimo-team/marimo-pair
```

Pair with the demo notebook:

```
/marimo-pair pair with me on Module_4/4_1_ai_features_demo.py
```

You can also pair from inside molab without any local install: open a molab notebook, click the actions panel, choose **Pair with an agent**. Useful for sandboxed runs and sharing finished work.

Source: https://docs.marimo.io/guides/generate_with_ai/marimo_pair/

2. MCP: tool context for notebooks

MCP is experimental, but it is important to mention because it explains how notebook-aware tools can be shared with agents and Chat.

marimo supports MCP in two directions:

- **MCP server**: exposes marimo notebook tools to external apps such as Claude Code, Cursor, or VS Code.
- **MCP client**: connects tools into marimo's Chat panel.

Start marimo with MCP support:

```
uv run --with="marimo[mcp]" marimo edit Module_4/4_1_ai_features_demo.py --
mcp --no-token
```

Or with `uvx`:

```
uvx "marimo[mcp]" edit Module_4/4_1_ai_features_demo.py --mcp --no-token
```

Important flags:

- `--mcp` exposes the marimo MCP server endpoint for external applications.
- `--no-token` disables authentication and should only be used for local development.
- If you only want marimo to act as an MCP client inside the Chat panel, install the MCP dependencies but omit `--mcp`.

Connect Claude Code to the running marimo MCP server:

```
claude mcp add --transport http marimo http://localhost:PORT/mcp/server
```

Example from a live local notebook:

```
claude mcp add --transport http marimo http://localhost:2720/mcp/server
```

Expected confirmation:

```
Added HTTP MCP server marimo with URL: http://localhost:2720/mcp/server to
local config
```

Then start Claude Code from the same project directory:

```
claude
```

Test prompts:

```
Use the marimo MCP server to list the active notebooks.
```

```
Use the marimo MCP tools to inspect the active notebook and tell me if
there are any cell errors.
```

If authentication is enabled, append `?access_token=YOUR_TOKEN` to the MCP server URL.

Enable useful MCP client presets in `marimo.toml`:

```
[mcp]
presets = ["marimo", "context7"]
```

Those preset tools are available in the Chat panel when using Ask mode. Custom MCP server configuration is not yet available in marimo.

Source: https://docs.marimo.io/guides/editor_features/mcp/

3. Skills and custom instructions

Skills are reusable instruction bundles for coding agents.

Install marimo's official skills:

```
npx skills add marimo-team/skills
```

Use skills for:

- creating marimo notebooks
- converting notebooks
- building custom widgets

Source: https://docs.marimo.io/guides/generate_with_ai/skills/
